

Prochaines étapes d'implémentation

Carnet numérique du troupeau — feuille de route par composant

Rappel : l'architecture et les interfaces sont en place dans les quatre dépôts. Il reste à implémenter la logique métier (corps en « Not implemented »/UnimplementedError) et à connecter les composants entre eux. Le service Python est déjà fonctionnel.

0. Préparatifs communs (avant de coder)

À faire une fois, en équipe, pour éviter les blocages d'intégration en fin de parcours.

- Figurer le contrat d'API entre app-mobile / admin-frontend et admin-service (routes, forme JSON) — le document « contrat d'API » sert de référence.
- Mettre en place un dépôt Git par composant (ou un monorepo) avec branches et revues de code.
- Préparer les données de terrain : 5–10 photos de mufles réels, valeurs nutritionnelles des aliments locaux, 2–3 trajets de transhumance, tuiles de carte de la zone de Ouagadougou.
- Définir les variables d'environnement de chaque service (URL API, jeton interne NestJS ↔ Python, base de données).

admin-service — NestJS (API générale, propriétaire de la base)

1. admin-service (NestJS)

C'est le composant central : rien ne fonctionne bout en bout tant que son API n'est pas vivante. À traiter en priorité.

1.1 Base de données & Prisma

1. Lancer prisma generate puis prisma db push à partir du schéma fourni (schema.prisma reflète la base serveur).
2. Vérifier la connexion via PrismaService et créer un jeu de données de départ (seed) : un admin, quelques éleveurs/agriculteurs, un référentiel d'aliments.

1.2 Adaptateurs de persistance

3. Implémenter les corps des repository adapters (animal, eleveur, alert, campaign, user, audit-log) : requêtes Prisma réelles.
4. Implémenter les mappers de persistance (ligne Prisma ↔ entité de domaine).

1.3 Cas d'usage (logique métier)

5. Enrôlement d'animal : appel interne au service Python (/internal/muzzle/enroll), stockage de la signature + fiche, génération du public-id préfixé, écriture d'audit.
6. Vérification : relais vers Python (/internal/muzzle/match), décision reconnu/volé/inconnu à partir du statut de l'animal correspondant.
7. Signalement de vol : passage du statut à « stolen », création du signalement, audit.
8. Ration : relais vers Python (/internal/ration/solve), historisation du résultat.
9. Alerte de conflit : identifier l'agriculteur de la zone et déclencher la notification (plans B/C, puis D en perspective).
10. Campagnes, liste des éleveurs, journal d'audit, paramètres.

1.4 Synchronisation & règle des 5 km

11. Endpoints /sync/push et /sync/pull : appliquer les événements idempotents (uuid client), gérer les conflits (le serveur est l'autorité).
12. GET /contacts/nearby : calculer les agriculteurs à ≤ 5 km de la route habituelle d'un berger (croiser traces et zones) et renvoyer les contacts filtrés.

1.5 Authentification & sécurité

- 13.PasswordService (hachage) et TokenService (JWT) ; use case login.
- 14.Vérification du jeton interne sur les appels sortants vers Python (en-tête X-Internal-Token).
- 15.Filtre d'erreurs métier : traduire BusinessException en réponses HTTP.

2. module-python (FastAPI)

Ce service est déjà fonctionnel (endpoints testés, 6 tests verts). Les tâches ci-dessous sont des améliorations, pas des prérequis au fonctionnement.

2.1 Fiabiliser la biométrie

- Calibrer le seuil de similarité (PY_MUZZLE_MATCH_THRESHOLD) sur de vraies photos de mufles.
- Évaluer le remplacement d'ORB par un extracteur MobileNet (meilleure robustesse) — sans changer le reste du service.

2.2 Affiner le solveur de ration

- Remplacer les besoins indicatifs par des références INERA/CIRDES par catégorie d'animal.
- Ajouter éventuellement des contraintes (minéraux, quantités mini par aliment).

2.3 Déploiement

- Construire l'image Docker fournie et exposer le service sur le réseau interne, joignable par NestJS.
- Activer le jeton interne (PY_INTERNAL_API_TOKEN) partagé avec NestJS.

3. app-mobile (Flutter)

Les écrans existent ; il reste à leur donner vie (données locales, caméra, GPS, synchro, SMS).

3.1 Base locale & couche données

16. Implémenter AppDatabase (ouverture SQLite, PRAGMA foreign_keys, création du schéma local fourni).
17. Implémenter les DAO (animals, rations, zones, sync) et les repositories.
18. Câbler AppServices.create() et AppScope (injection de dépendances).

3.2 Anti-vol

19. Accès caméra et capture des photos de mufle (enrôlement, vérification).
20. Enrôlement : écriture locale + mise en file d'attente (outbox) pour la synchro.
21. Vérification : comparaison sur le cache local de signatures ; sinon « à vérifier en ligne ».
22. Signalement de vol : passage du statut local et file de synchro.

3.3 Ration

23. Saisie du troupeau et des aliments ; appel du calcul (serveur en ligne, ou solveur local léger si hors ligne).
24. Affichage et historisation locale du résultat.

3.4 Carte & alertes

25. Intégrer la carte hors ligne (tuiles pré-téléchargées) et l'affichage des couloirs/zones.
26. Suivi GPS et détection d'approche d'une zone cultivée (seuil configurable).
27. Alerte locale au berger + cascade : SMS direct (plan A) via le cache 5 km, sinon notification serveur (B/C).
28. Demander la permission SMS et implémenter l'envoi natif.

3.5 Synchronisation & réglages

29. SyncService : pousser l'outbox et récupérer les changements (contacts 5 km, vols, zones).
30. Écrans Réglages : profil, code d'agent, langue, état de synchro.

4. admin-frontend (Next.js)

Les pages et composants existent avec des données de démonstration ; il reste à les brancher sur l'API.

4.1 Connexion à l'API

31. Implémenter le client API (lib/api/client.ts) et les endpoints réels.
32. Authentification : formulaire de login relié à /auth/login, gestion de session et middleware de protection des routes.

4.2 Brancher les pages sur les vraies données

- Tableau de bord : statistiques réelles (animaux, vols, conflits, troupeaux actifs).
- Suivi des troupeaux : positions et trajets agrégés depuis l'API.
- Alertes & litiges : liste et changement de statut (résolu / faux).
- Sensibilisation : création et suivi des campagnes.
- Utilisateurs, audit, paramètres : lecture/écriture via l'API.

4.3 Aide à la décision — marché

- Intégrer la carte réelle (heatmap de convergence) et l'export en PDF/image.

5. Intégration de bout en bout

33. Chaîne anti-vol complète : app-mobile → admin-service → module-python → retour, en local (Wi-Fi partagé) et hors ligne (mode avion).
34. Chaîne d'alerte de conflit : détection GPS → SMS direct (plan A) et notification serveur (B/C) → visible dans admin-frontend.
35. Passerelle SMS : brancher un vieux téléphone Android (ou dongle GSM) pour envoyer de vrais SMS pendant la démo.
36. Préparer la démo : mufles pré-enregistrés, vidéo de secours, scénario chronométré (3–5 min).

6. Ordre de priorité recommandé

Pour livrer une démo qui marche, dans cet ordre :

Ordre	Tâche	Dépend de
1	admin-service : base + persistance + enrôlement/vérification + auth	module-python (déjà prêt)
2	app-mobile : base locale + anti-vol (caméra, enrôlement, vérif)	admin-service 1
3	Intégration chaîne anti-vol (app → serveur → python)	1 et 2
4	app-mobile : ration + carte + alertes + synchro	admin-service
5	admin-frontend : login + tableau de bord + alertes	admin-service
6	admin-service : sync, contacts 5 km, campagnes, audit	—